

the case of urllib, where you need to use the method `urllib.encode()` to encode the parameters before passing them.

- Python Requests automatically decodes the response into Unicode.
- Python Requests automatically saves the contents, enabling you to access it multiple times, unlike the read-once file-like object returned by `urllib2.urlopen()`.
- Python Requests handles multi-part file uploads, as well as automatic form-encoding.
- In Python, `Requests.get()` is a method, `auth` is an optional parameter (since we may or may not require authentication).
- Python Requests supports the entire restful API, i.e., all its methods - PUT, GET, DELETE, POST.
- Python Requests has a built-in JSON decoder.
- Unlike the `urllib/urllib2` module, there is no confusion caused by Requests, as there is only a single module that can do the entire task.
- Can write easier and shorter code.

## A comparison of Python Requests and urllib/urllib2

Here are some simple examples using which you can easily make a comparison between Python Requests and `urllib/urllib2`.

### Example 1: A simple HTTP GET request and authentication

*Using urllib2:* In this example, to make a simple HTTP GET request we need to call a lot of methods. Remembering the names of these methods can be difficult:

```
import urllib2

url = 'https://www.example.com'
username= 'user'
password = 'pass'

request = urllib2.Request(url)

password_manager = urllib2.HTTPPasswordMgrWithDefaultRealm()
password_manager.add_password(None, url, username, password)

auth_manager = urllib2.HTTPBasicAuthHandler(password_manager)
opener = urllib2.build_opener(auth_manager)

urllib2.install_opener(opener)

handler = urllib2.urlopen(request)

print handler.getcode()
print handler.headers.getheader('content-type')
```

*Using Requests:* The task of making a simple HTTP GET request can be accomplished in a single line when

compared to the large code written using `urllib2`.

```
import requests

r = requests.get('https://www.example.com', auth=('user',
'pass'))

print r.status_code
print r.headers['content-type']
```

### Example 2: Making a POST request

*Using urllib2/urllib:* Note that in this example we had to make use of both the `urllib` and `urllib2` modules in order to write a script for a simple POST request:

```
import urllib
import urllib2

url = "http://www.example.com"
values = {"firstname":" abc ", "lastname":" xyz "}

header = {"User-Agent":"Mozilla/4.0 (compatible; MSIE
5.5;Windows NT)"}

values = urllib.urlencode(values)
request = urllib2.Request(url, values, header)

response = urllib2.urlopen(request)
html_content = response.read()
```

*Using Requests:* Here we do not require import multiple modules and a single requests module can accomplish the entire task:

```
import requests

values = {"firstname":" abc ", "lastname":" xyz "}
r = requests.post('https://www.example.com, data=values')
```



## Acknowledgments

I would like to thank Kenneth Reitz, lead developer of Python Requests, for designing and developing the Python Requests library. I would also like to thank my mentor and all the people who helped me to review this article.

## References

- [1] <http://docs.python-requests.org/en/latest/>
- [2] <https://pypl.python.org/pypl/requests>

## By: Sakshi Bansal

The author is a FOSS enthusiast and an active member of Amrita FOSS club. She has contributed to various open source projects such as Mozilla Thunderbird, Mediawiki. She blogs at <http://sakshiil.wordpress.com/>.